

KeepClip — dokumentacja frontendu

Interfejs w WebView2: struktura strony, widoki, odtwarzacz, samouczek, tłumaczenia

Wersja aplikacji 2.3.0 · gałąź `csharp` · stan na 17 września 2026

Interfejs KeepClip to zwykła strona internetowa bez frameworka: jeden plik HTML, dwa pliki skryptów, dwa arkusze stylów i słownik tłumaczeń. Ładuje ją kontrolka WebView2 (silnik Edge/Chromium) w oknie WinForms. Wszystkie dane pochodzą z lokalnego serwera HTTP opisanego w dokumencie *Backend*. Ten dokument opisuje, jak strona jest zbudowana, co robi każdy widok i jak ją rozszerzać.

Spis treści

1. Pliki i ich role
2. Okno i most do aplikacji
 - 2.1 Okno bez ramki
 - 2.2 Interfejs `window.pywebview.api`
 - 2.3 Klawisz F10
3. Start strony i wykrywanie języka
 - 3.1 Skrypt wbudowany (uruchamia się przed `app.js`)
 - 3.2 `app.js`: funkcja `bootstrap()`
 - 3.3 Zegary działające cały czas
4. Układ strony i nawigacja
5. Widok Klipy
 - 5.1 Lista i wyszukiwanie
 - 5.2 Gwiazdka, chmura, zaznaczanie
 - 5.3 Podgląd po najechaniu
6. Widok Foldery i tryb zaznaczania
7. Widok Ulubione i Chmura
8. Odtwarzacz
 - 8.1 Otwieranie (`openPlayer(clipld, startAt)`)
 - 8.2 Sterowanie
 - 8.3 Edycja wersu
 - 8.4 Akcje boczne
9. Modal wycinania
10. Transkrypcja zbiorcza i paski postępu
11. Okno powtórek i przechwytywanie skrótu
12. Modal konfiguracji folderu i ustawienia
13. Potwierdzenia, toasty, błędy
14. Aktualizacje w interfejsie
15. Samouczek (`onboarding.js`)
 - 15.1 Idea
 - 15.2 Silnik
 - 15.3 Podpowiedzi o nowych funkcjach
16. Tłumaczenia (`i18n.js`)
17. Style
18. Pamięć przeglądarki
19. Panel deweloperski
20. Jak rozszerzać interfejs

1. Pliki i ich role

Plik	Rozmiar	Rola
frontend/index.html	ok. 1050 wierszy	Cała struktura DOM: pasek tytułu, pasek boczny, pięć widoków, nakładki (odtwarzacz, wycinanie, konfiguracja, powtórki, potwierdzenie, logi), toast, baner aktualizacji. Na końcu skrypt wbudowany z wykrywaniem języka i funkcją <code>t()</code> , potem <code>app.js</code> , <code>onboarding.js</code> i drugi skrypt wbudowany (animowane tło, animacja przyciągania okna).
frontend/app.js	ok. 3500 wierszy	Cała logika interfejsu (bez samouczka). Skrypt klasyczny, bez modułów; funkcje i stan w zakresie globalnym.
frontend/onboarding.js, onboarding.css	ok. 990 + 190 wierszy	Silnik samouczka (reflektor na elementach interfejsu) i podpowiedzi o nowych funkcjach. Udostępnia <code>window.KeepClipOnboarding</code> .
frontend/i18n.js	ok. 2180 wierszy	Obiekt <code>i18n</code> z czterema językami (<code>pl</code> , <code>ru</code> , <code>en</code> , <code>uk</code>), po 427 kluczy każdy. Eksportowany też jako moduł CommonJS dla skryptu kontrolnego.
frontend/styles.css	ok. 1970 wierszy	Motyw (zmienne CSS), układ, komponenty, odtwarzacz, widoki, panel logów.
frontend/icons/, frontend/sounds/		Ikony (favicon w dwóch kolorach, ikona aplikacji <code>icon.ico</code>), dźwięki potwierdzenia zapisu powtórki.

Serwer podaje te pliki pod `/static/...` z dopiskiem `?v=<czas modyfikacji>` w `index.html`, więc po aktualizacji przeglądarka zawsze pobiera nową wersję. Bloki między znacznikami `DEV:START` / `DEV:END` (panel logów w HTML, JS i CSS) są wycinane z wydania publicznego przez `installer/build.ps1`. Te znaczniki to jedyne komentarze w kodzie interfejsu i muszą zostać.

2. Okno i most do aplikacji

2.1 Okno bez ramki

Formularz WinForms ma `FormBorderStyle.None`, więc pasek tytułu rysuje strona (`#custom-titlebar`). Przeciąganie działa dzięki klasie `pywebview-drag-region`: skrypt mostu (wstrzykiwany przez C# do każdej strony) przechwytyje `mousedown` na takim elemencie i prosi okno o natywne przesuwanie (`WM_NCLBUTTONDOWN` z `HTCAPTION`). Krawędzie okna to niewidoczne uchwyty dodawane przez most (`data-keepclip-grips`), które zgłaszają kody trafienia Win32 do skalowania. Nowsze WebView2 obsługuje też regiony natywne (`IsNonClientRegionSupportEnabled`).

2.2 Interfejs `window.pywebview.api`

Nazwa pochodzi z wcześniejszej wersji aplikacji w Pythonie; most C# odtwarza to samo API, więc strona nie musiała się zmieniać. Każde wywołanie to `postMessage` z `{__kc:true, id, method, args}` i odpowiedź `{__kcReply:true, id, result, error}`.

Metoda	Działanie
<code>pick_folder()</code>	Systemowe okno wyboru folderu; zwraca ścieżkę albo <code>null</code> .
<code>minimize_window()</code> , <code>close_window()</code>	Przyciski paska tytułu. Zamknięcie przy włączonym nagrywaniu w tle chowa okno do zasobnika zamiast kończyć proces.
<code>toggle_maximize_window(w, h, left, top)</code>	Maksymalizacja do obszaru roboczego ekranu podanego przez stronę (<code>screen.avail*</code>) i powrót.
<code>start_native_resize()</code>	Skalowanie za prawy dolny róg.

Funkcja `callWindowApi(method, ...args)` w `app.js` opakowuje te wywołania i loguje brak mostu (np. w zwykłej przeglądarce w trybie serwera). Most wysyła też zdarzenie `pywebviewready`.

2.3 Klawisz F10

Windows traktuje F10 jako aktywację paska menu i WebView2 nie przekazuje go stronie. Okno przechwytyje F10 w `ProcessCmdKey` i wstrzykuje zdarzenie `keydown` do aktywnego elementu strony, dzięki czemu pole skrótu powtórki może ustawić np. `Alt+F10`.

3. Start strony i wykrywanie języka

3.1 Skrypt wbudowany (uruchamia się przed app.js)

1. Język: parametr `?lang=` w adresie (zapisywany do `localStorage.keepclip_lang`) → zapisany w `localStorage` → język systemu (`navigator.language`) → `en`.
2. Ustawia `window.LANG`, atrybut `lang` dokumentu i globalną funkcję `window.t(klucz)`, która schodzi po kropkach w obiekcie `i18n[lang]` i zwraca sam klucz, gdy tłumaczenia brak.
3. Stosuje atrybuty: `data-i18n` (tekst), `data-i18n-html`, `data-i18n-title`, `data-i18n-aria`, `data-i18n-placeholder`; ustawia podpisy przycisków odtwarzacza, placeholder wyszukiwarki, opcje sortowania.
4. Obsługuje listę `#lang-select`: zapisuje wybór, zgłasza go do `POST /api/language` (od tego zależy język transkrypcji) i przeładowuje stronę z `?lang=`.

3.2 app.js: funkcja bootstrap()

1. `GET /api/config` → wersja w ustawieniach (`KeepClip • v2.3.0`); jeśli język serwera różni się od języka strony, zgłasza język strony; jeśli `configured=false`, otwiera modal wyboru folderu w trybie *pierwszego uruchomienia* (bez przycisku Anuluj) i kończy start — resztę uruchomi zapis folderu.
2. `POST /api/scan` (cichy), `GET /api/stats`.
3. `GET /api/cloud/status` → oznaczenie połączenia, a gdy połączono — cicha synchronizacja z Dyskiem.
4. `checkForUpdate()` (baner, gdy jest nowsze wydanie).
5. `GET /api/transcribe/status` → jeśli trwa transkrypcja, podpiną strumień postępu.
6. `renderRecent("")` → lista klipów, potem `startAutoLibrarySync()`.

3.3 Zegary działające cały czas

Co	Interwał	Uwagi
<code>POST /api/heartbeat</code>	30 s	Bez tego serwer zamknąłby się po 5 minutach.
<code>autoSyncLibrary</code> (skan + statystyki)	2,5 s	Pomijane, gdy karta jest ukryta lub otwarty jest modal konfiguracji; odświeża widoki tylko, gdy statystyki się zmieniły. Dodatkowo przy powrocie fokusu do okna.
<code>refreshReplayStatus</code>	15 s (3 s przy otwartym oknie powtórek)	Kropka przy przycisku „Powtórka” w pasku bocznym.
Wyszukiwanie	200 ms po ostatnim znaku	Debounce pola <code>#q</code> .
Podgląd po najechaniu	250 ms	Dopiero po tym czasie karta prosi o źródło wideo.
Wskaźnik aktywnego wersu	300 ms	Podświetla wers transkrypcji zgodny z czasem odtwarzania.
Postęp pojedynczej operacji	300 ms	<code>/api/single-operation/status</code> w czasie wycinania lub transkrypcji jednego klipu.
Łączenie z Google	2 s, do 150 prób	Odpytuje <code>/api/cloud/status</code> po otwarciu zgody w przeglądarce.
Panel logów	1,5 s	Tylko build deweloperski.

4. Układ strony i nawigacja

```
#custom-titlebar (przeciąganie, _ □ ×)
.app-shell
├─ aside.sidebar (#keepclip-sidebar, zwiżany przyciskiem #sidebar-toggle)
│   └─ logo · Główne: Klipy / Foldery / Ulubione / Chmura (kropka gdy połączono)
│       └─ Narzędzia: Skanuj (#btn-scan) · Transkrybuj (#btn-transcribe) · Transkrybuj ponownie wszystko
│           └─ Zmień folder · Powtórka (#btn-replay, kropka gdy nagrywa) · Logi (DEV) · Ustawienia (#btn-settings)
│               └─ statystyki (#stat-clips, #stat-trans, #stat-fav), pasek miejsca lokalnie / w chmurze,
│                   └─ mini-karta konta Google (#sidebar-cloud-card), pasek postępu transkrypcji (#progress)
└─ main
    └─ #view-clips      wyszukiwarka #q, filtr #game-filter, sortowanie #sort, przełącznik układu, siatka #results, pasek zaznaczania
        └─ #view-folders #folders-index (kafle) / #folder-detail (klipy folderu, przyciski)
            └─ #view-favorites #fav-results
                └─ #view-cloud      #cloud-loading / #cloud-setup / #cloud-connect / #cloud-account
                    └─ #view-settings język, motyw, animowane tło, folder klipów, samouczek, wersja
                        └─ nakładki: #player-overlay, #cut-overlay, #config-overlay, #replay-overlay, #confirm-overlay, #logs-overlay
                            └─ inne: #toast, #single-progress, #update-banner, #snap-hint
```

`setView(nazwa)` przełącza atrybut `hidden` widoków i klasę `active` w pasku bocznym; wejście do Folderów, Ulubionych i Chmury ładuje ich dane. Ustawienia to pełny widok, nie modal. Stan zwinięcia paska bocznego jest pamiętany w `localStorage.keepclip_sidebar_collapsed`; w zwiniętym trybie przyciski dostają tooltipy z własnych etykiet.

5. Widok Klipy

5.1 Lista i wyszukiwanie

- `doSearch()`: puste pole → `renderRecent(gra, sort)` pobiera `GET /api/clips?sort&limit=120[&game]`. Z tekstem → `GET /api/search?q&sort&limit=200` i filtrowanie po grze po stronie strony. Opcja sortowania „trafność” jest widoczna tylko z zapytaniem (`syncSortOptions`).
- Kartę klipu buduje `clipCard(c)`: miniatura `/thumb/{id}?v=rozmiar` (z zastępczą ikoną przy błędzie), czas trwania, znaczek chmury (`cloudChip`), pole zaznaczenia w trybie wyboru, gra, data (`fmtDate`), gwiazdka (`favStar`), rozmiar i informacja „nie transkrybowano”. Atrybuty `data-size` i `data-duration` służą do szacowania rozmiaru wycinka.
- Wyniki wyszukiwania (`renderResults`) pokazują dodatkowo czas trafienia i fragment z podświetleniem (`safeSnippet` koduje HTML poza `<mark>`). Kliknięcie otwiera odtwarzacz od miejsca trafienia (0,5 s wcześniej).
- Układ: `grid` albo `mosaic` (`applyLayout`, pamiętane w `localStorage.keepclip_layout`, domyślnie mozaika). W mozaice ulubione idą na początek. `_repaint` pozwala przerysować bieżącą listę bez ponownego pobierania.

5.2 Gwiazdka, chmura, zaznaczanie

Trzy nasłuchcy na fazie przechwytywania (`addEventListener(..., true)`) obsługują kliknięcia w gwiazdkę, znaczek chmury i kartę w trybie zaznaczania, zanim dojdzie do otwarcia odtwarzacza. `toggleFavorite` synchronizuje wszystkie karty tego klipu w DOM. `uploadClipToCloud` / `downloadClipFromCloud` blokują powtórne kliknięcia (`_busyClipIds`), podmieniają znaczek po sukcesie i odświeżają statystyki oraz limit chmury.

5.3 Podgląd po najechaniu

Po 250 ms na karcie strona pyta `GET /api/playback/{id}?start=false`. Jeśli źródło jest gotowe (klip ≤ 120 FPS albo gotowa kopia podglądowa), wstawia wyciszony element `<video>` z fragmentem `#t=...` (start od trafienia, od połowy krótkiego klipu albo od 5 s) i klasą `previewing`. Opuszczenie karty, przewinięcie lub tryb zaznaczania zatrzymują podgląd. Token `_previewToken` chroni przed spóźnionymi odpowiedziami.

6. Widok Foldery i tryb zaznaczania

- `loadFoldersIndex()`: kafle z kolażem do 4 miniatur ostatnio dodanych klipów, liczbą klipów i kaflem „nowy folder” (`promptNewFolder` używa `window.prompt`).
- `openFolder(id)`: `GET /api/folders/{id}/clips?sort=added`, karty jak w Klipach; przyciski: wstecz, zmień nazwę (`PATCH`), dodaj klipy, wyślij folder do chmury (widoczny tylko po połączeniu), usuń (z potwierdzeniem).
- **Tryb zaznaczania** (`startSelectionMode`): przenosi do widoku Klipy z paskiem `#selection-bar`; klikanie kart przełącza zaznaczenie i synchronizuje wszystkie karty tego samego klipu; „Dodaj” wysyła równoległe `POST /api/folders/{f}/clips/{c}` dla każdego zaznaczenia i wraca do folderu.
- Z odtwarzacza foldery obsługuje rozwijane menu `#player-folders-dropdown` (pola wyboru + tworzenie folderu w miejscu). Menu jest przenoszone do `body` (portal), żeby nie przycinał go odtwarzacz; pozycję liczy `positionFoldersDropdown`.

7. Widok Ulubione i Chmura

Ulubione: `GET /api/clips?favorite=1&limit=500`, liczba w nagłówku, te same karty.

Chmura (`loadCloud`) pokazuje jeden z paneli: ładowanie, *konfiguracja* (brak pliku klienta OAuth — instrukcja tworzenia własnego klienta; w instalatorze plik jest wbudowany, więc użytkownicy tego nie widzą), *połącz* (przycisk startujący `startCloudConnect`), *konto* (nazwa, e-mail, zdjęcie przez `/api/cloud/avatar` z inicjałami jako zapasem, pasek zajętości, rozłącz z potwierdzeniem). `startCloudConnect` otwiera adres zgody w nowej karcie (`WebView2` przekazuje go do przeglądarki systemowej) i odpytuje status co 2 s; po połączeniu synchronizuje bibliotekę. Powrót fokusu do okna też odświeża panel. Paski miejsca w pasku bocznym (`renderStorage`) łączą statystyki lokalne z limitem Dysku; minimalna szerokość 0,5 % pozwala zobaczyć nawet mały udział klipów.

8. Odtwarzacz

8.1 Otwieranie (`openPlayer(clipId, startAt)`)

1. Zatrzymuje podgląd, pobiera `GET /api/segments/{id}`. Klip w chmurze bez połączenia → toast i koniec.
2. Wypełnia nagłówkę, gwiazdkę, przycisk chmury (`updatePlayerCloudBtn`: strzałka w dół dla klipu w chmurze, w górę dla lokalnego, ukryty bez połączenia), plakat z miniatury (zapobiega czarnemu ekranowi), listę wersów z przyciskiem  i przycisk „Pobierz .txt” (tylko gdy są segmenty).
3. Buduje napisy WebVTT (`rebuildSubtitles`) i pokazuje nakładkę.
4. `resolvePlaybackSource` odpytuje `GET /api/playback/{id}` co 500 ms, pokazując pasek „Przygotowuję podgląd” z procentem dla klipów powyżej 120 FPS, aż dostanie adres; potem ustawia `video.src`, przewija do `startAt - 0,5 s` i odtwarza. Token `_playbackRequestToken` unieważnia poprzednie oczekiwanie przy szybkiej zmianie klipu.

8.2 Sterowanie

Własne przyciski (`#ctrl-*`): zrzut ekranu do JPEG (canvas), głośność w wyskakującym suwaku, początek/-5 s/odtwarzaj/+5 s/koniec, prędkość w cyklu 1 → 1,25 → 1,5 → 2 → 0,5 → 0,75 (nowy klip zawsze od 1×), napisy (stan w `localStorage.kc_subtitles`), pełny ekran całego `#player-box` (sterowanie znika po 2,5 s bezczynności, wraca przy pauzie). Spacja przełącza odtwarzanie, Esc zamyka (w pełnym ekranie pierwsze Esc obsługuje przeglądarka). Kliknięcie w wers przewija wideo; aktywny wers jest podświetlany i przewijany do widoku, chyba że trwa edycja.

8.3 Edycja wersu

`enterSegEdit` zamienia wers w pole tekstowe (Ctrl+Enter zapisuje, Esc anuluje), wysyła `PATCH /api/segments/{id}`, aktualizuje tekst, napisy i pokazuje toast. W czasie edycji Esc nie zamyka odtwarzacza.

8.4 Akcje boczne

Przycisk	Co robi
Ulubione	<code>POST /api/clips/{id}/favorite</code> .
Wytnij	Otwiera modal wycinania (niżej).
Chmura	Wysyła lub ściąga klip; w trakcie etykieta „Wysyłam.../Pobieram...”.
Foldery	Rozwijane menu członkostwa.
Transkrypcja	<code>POST /api/clips/{id}/retranscribe</code> z paskiem postępu; po sukcesie odtwarzacz otwiera się ponownie.
Pobierz .txt	<code>HEAD</code> na <code>/api/clips/{id}/transcript.txt</code> (czytelny błąd zamiast pustego pliku), potem kliknięcie ukrytego odnośnika z <code>download</code> .
Napraw	Potwierdzenie, <code>POST /api/clips/{id}/fix</code> , toast z liczbą przyciętych sekund.
Usuń	Potwierdzenie (inne teksty dla klipu w chmurze), zamknięcie odtwarzacza i 200 ms przerwy (zwolnienie uchwytu pliku), <code>DELETE /api/clips/{id}?delete_file=true</code> .

9. Modal wycinania

`openCutModal` kopiuje źródło wideo z odtwarzacza do `#cut-video`, ustawia zakres domyślnie od bieżący czas – 5 s na 10 s. Suwak zakresu (`#cut-range`) ma dwa uchwyty i wskaźnik odtwarzania; przeciąganie uchwyty przewija wideo do jego czasu, kliknięcie w tor przesuwania bliższy uchwyt. Presety rozmiaru: bez kompresji, 20 MB (Discord), 25, 50 (Nitro Basic), 100, 500 MB (Nitro) albo własna liczba. `updateCutEstimate` szacuje wynik: bez kompresji z przepływności pliku źródłowego (rozmiar/czas z atrybutów karty), z limitem odtwarza obliczenie serwera (limit – 100 kB – audio 128 kb/s – 50 kB → przepływność wideo) i opisuje jakość: < 1500 kb/s słaba, < 3000 średnia, < 6000 dobra, wyżej świetna. Przycisk „Wytnij” wysyła `POST /api/clips/{id}/cut` z paskiem postępu; po sukcesie `showCutSuccess` pokazuje nazwę i rozmiar oraz przycisk otwierający folder w Eksploratorze (`POST /api/show-in-explorer`). Modal ma własny mini-odtwarzacz (skrypt na końcu `app.js`).

10. Transkrypcja zbiorcza i paski postępu

- `startTranscribe(force)`: najpierw `POST /api/scan`, potem `POST /api/transcribe/start`. Brak nowych klipów → toast. Sukces → `attachStream()`.
- `attachStream`: `EventSource("/api/transcribe/stream")`; każda ramka aktualizuje pasek `#progress` (`done / total`, bieżący klip). Koniec strumienia odświeża statystyki i listę. Błąd strumienia przełącza na odpytywanie `/api/transcribe/status` co 1,5 s (`pollOnce`).
- „Transkrybuj ponownie wszystko” pokazuje potwierdzenie z liczbą klipów i używa `force=true`.
- Pasek `#single-progress` (`startSingleOperationProgress`) obsługuje wycinanie i transkrypcję jednego klipu; identyfikator operacji chroni przed pomyleniem z poprzednią; błąd zostawia pasek z komunikatem i przyciskiem zamknięcia.

11. Okno powtórek i przechwytywanie skrótów

`openReplayModal` pobiera status, wypełnia formularz (włącz, w tle, mikrofon, długość, FPS, jakość, skrót, urządzenia audio z `/api/replay/audio-devices` albo z `navigator.mediaDevices` jako zapas) i odpytuje status co 3 s. `renderReplayStatus` opisuje stan (kodowanie: NVENC/AMF/QuickSync/x264, „bez audio”), ostrzega, gdy bufor działa, ale skrót nie jest aktywny (konflikt z inną aplikacją). „Zapisz ustawienia” → `POST /api/replay/config`; „Zapisz teraz” → `POST /api/replay/save`.

Przechwytywanie skrótów: kliknięcie w pole przełącza je w tryb nasłuchu; `winFormsKeyName(e)` tłumaczy `e.code` na nazwy z `System.Windows.Forms.Keys` (litery, `D0-D9`, `NumPad*`, `F1-F24`, klawisze `Oem*` dla interpunkcji, `Back` zamiast `Backspace`). Wymagany jest co najmniej jeden modyfikator (`Ctrl/Alt/Shift/Win`); sam klawisz przechwytywałby pisanie w całym systemie. `Esc` przywraca poprzedni skrót.

12. Modal konfiguracji folderu i ustawienia

`openConfigModal({mode:"first"|"change"})` różni się tytułem i brakiem Anuluj przy pierwszym uruchomieniu. Przycisk folderu używa `pywebview.api.pick_folder`. Zapis → `POST /api/config`; sukces zamyka modal, pokazuje wynik skanu, odświeża widoki, uruchamia auto-sync i wysyła zdarzenie `keepclip:configured`, na które czeka samouczek.

Ustawienia: język (przeładowanie strony), motyw (`POST /api/accent-theme`, natychmiastowa zmiana `data-accent-theme` i favicon, cofnięcie przy błędzie), animowane tło (przełącznik w drugim skrypcie wbudowanym; kule `#orb1/#orb2` dostają klasę `paused`), zmiana folderu, uruchomienie samouczka, wersja.

13. Potwierdzenia, toasty, błędy

- `showConfirm({title, body, warning, warningClass, yesLabel, yesClass})` zwraca `Promise<bool>`; Esc i kliknięcie tła = odmowa; domyślne ostrzeżenie mówi o Koszu Windows.
- `toast(msg, "ok"|"error")` — 3,5 s w prawym dolnym rogu.
- `backendErrorMessage(payload, fallback)`: gdy serwer podał kod `error` i istnieje klucz `errors.<kod>`, używa tłumaczenia; inaczej `detail` (z jedną dodatkową regułą w `localizeBackendError` dla błędu dekodowania audio).

14. Aktualizacje w interfejsie

`checkForUpdate` woła `GET /api/update/check`; przy nowszej wersji pokazuje `#update-banner` z numerem (odrzuć na czas sesji w `sessionStorage.kc_upd_dismiss`). „Pobierz i zainstaluj” → `POST /api/update/install` i odpytywanie `/api/update/status` co 500 ms z procentem; faza `starting` oznacza, że aplikacja zaraz się zamknie, a instalator podmieni pliki w miejscu.

15. Samouczek (onboarding.js)

15.1 Idea

Samouczek nie ma własnych ekranów: przyciemnia aplikację, wycina „reflektor” wokół prawdziwego elementu i pokazuje obok kartę z tekstem. Kroki są deklaratywne (`STEPS`), elementy odnajduje rejestr `TARGETS` (lista selektorów, wygrywa pierwszy widoczny, więc krok działa też przy pustej bibliotece).

Krok	Widok	Cel	Warunek
welcome	—	karta na środku	
automatic-clips	clips	biblioteka <code>#results</code>	tylko gdy folder ustawiony
clips-folder	settings	przycisk zmiany folderu	tylko gdy folder nieustawiony
games-organized	clips	<code>#game-filter</code>	
clip-editing	clips	pierwsza karta klipu	
transcription	clips	<code>#btn-transcribe</code>	
transcript-edit	clips	karta klipu / nawigacja	
favorites	clips	gwiazdka / nawigacja Ulubione	
collections	folders	nowy folder / siatka	
cloud	cloud	przycisk łączenia / karta konta	
done	clips	karta na środku	

15.2 Silnik

- **Nakładka:** `.kc-ob-veil` (blokuje klikanie; zaczyna 32 px od góry, żeby pasek tytułu dalej działał), `.kc-ob-spot` (cień 9999 px rysuje przyciemnienie z otworem; element wyższy niż 40 % okna dostaje reflektor tylko u góry), `.kc-ob-card` (karta, tryb `kc-ob-modal` dla powitania i zakończenia).
- **Przejście** (`show(i)`): 150 ms przerwy, przełączenie widoku przez kliknięcie prawdziwego przycisku nawigacji (żeby wykonała się logika widoku), oczekiwanie na cel do 1,4 s (z 0,5 s karencji, gdy widać tylko cel zapasowy), przewinięcie, render karty, zapis `current_step`. Otwarte okna modalne aplikacji (odtwarzacz, wycinanie, konfiguracja, powtórki, potwierdzenie, edycja wersu) wstrzymują automatyczną nawigację.
- **Śledzenie:** pętla `requestAnimationFrame` przelicza pozycję reflektora i karty przy przewijaniu, zmianie rozmiaru i podmianie DOM; zniknięcie celu przenosi kartę na środek bez reflektora (samouczek nigdy nie przerywa się z tego powodu).
- **Karta:** wybiera pierwszą stronę z miejscem (preferowana, potem prawo, dół, lewo, góra), strzałka tylko gdy karta stoi obok celu; klawiatura: Enter/→ dalej, ← wstecz, Esc pytanie o pominięcie, Tab zamknięty w karcie.
- **Stan:** `GET/POST /api/onboarding`; gdy serwer nie odpowiada (stary backend), zapas w `localStorage` tylko na czas sesji i bez automatycznego startu (port jest losowy, więc `localStorage` nie przetrwa restartu).
- **Start:** po załadowaniu strony, jeśli nie ukończono i nie pominięto: z zapisanym krokiem → pytanie „Kontynuuj / Od początku / Pomiń”, inaczej od początku. Świeża instalacja czeka na zdarzenie `keepclip:configured`. Przycisk w ustawieniach woła `POST /api/onboarding/restart`.

15.3 Podpowiedzi o nowych funkcjach

```
KeepClipOnboarding.registerFeatureHint({
  id: "share-v1", target: "ShareButton", placement: "left",
  title: "Udostępnianie", description: "Możesz teraz wysłać klip jednym kliknięciem."
});
```

Cel musi istnieć w `TARGETS`. Podpowiedzi pokazują się tylko osobom, które ukończyły lub pominęły samouczek; zobaczone identyfikatory trafiają do `seen_hints`. Nowa funkcja nie uruchamia całego samouczka od nowa i nie wymaga podbicia `VERSION` (który musi zgadzać się z `OnboardingService.CurrentVersion`).

API publiczne: `start(stepId)`, `restart()`, `stop()`, `isActive()`, `state()`, `registerFeatureHint()`, `showPendingHints()`, `targets`, `steps`. Teksty kroków to klucze `onboarding.steps.<id>.{title, body, hint, note, primary, secondary}`; brakujący tekst po prostu znika z karty.

16. Tłumaczenia (i18n.js)

- Cztery języki, identyczny zestaw kluczy. Grupy: `nav`, `replay`, `orb`, `errors`, `progress`, `clipCount`, `selection`, `search`, `clip`, `player`, `playerActions`, `toast`, `confirm`, `cutSuccess`, `cutEstimate`, `storage`, `cloud`, `folders`, `folderDetail`, `config`, `titlebar`, `plural`, `stat`, `storageTitle`, `storageLabel`, `cloudPanel`, `cloudx`, `cutOverlay`, `confirmDefault`, `configLabel`, `ui`, `update`, `settings`, `onboarding`.
- Symbole w tekstach: `{n}`, `{word}`, `{error}`, `{path}`... podstawiane przez `.replace`. Angielski celowo nie ma `{word}` w kilku kluczach (`clip(s)`).
- Liczebniki: `clipsWord(n)` używa `Intl.PluralRules` i kluczy `clipCount.one/few/many/other`.
- Kontrola: `node tests/i18n-check.js` sprawdza komplet kluczy, puste teksty i klucze użyte w kodzie bez tłumaczenia; różnice w symbolach są ostrzeżeniem. Uruchamia się w CI.
- Dodając tekst: klucz w czterech językach, użycie przez `t("grupa.klucz")` albo atrybut `data-i18n` w HTML.

17. Style

Motyw definiują zmienne w `:root`: tła (`--bg`, `--card`, `--card-2`), teksty (`--text`, `--muted`), akcenty (`--accent`, `--accent-primary`, `--accent-vivid`, `--accent-2`, `--accent-folder`, `--accent-cloud`, `--accent-pale`, `--accent-stat` ...) oraz składowe RGB do przezroczystości (`--accent-rgb`, `--accent-border-rgb`). Motyw zielony nadpisuje je selektorem `:root[data-accent-theme="green"]`, więc nowe komponenty powinny używać zmiennych, a nie kolorów wprost — dzięki temu działają w obu motywach (samouczek korzysta wyłącznie ze zmiennych). Szerokość paska bocznego: `--sidebar-width 280 px` / `--sidebar-collapsed-width 92 px`. Elementy z klasą `pywebview-drag-region` nie mogą zawierać przycisków, bo pasek tytułu przejmuje mysz. Rozmiar napisów WebVTT zostaje domyślny, żeby skalował się w pełnym ekranie.

18. Pamięć przeglądarki

Klucz	Gdzie	Znaczenie
keepclip_lang	localStorage	Język interfejsu.
keepclip_sidebar_collapsed	localStorage	Zwinięty pasek boczny.
keepclip_layout	localStorage	grid / mosaic.
kc_subtitles	localStorage	Napisy włączone.
keepclip_onboarding	localStorage	Zapas stanu samouczka, gdy serwer nie ma endpointu.
kc_upd_dismiss	sessionStorage	Odrzucony baner aktualizacji (wersja).

Okno aplikacji dostaje losowy port przy każdym starcie, a `localStorage` jest przypisany do adresu razem z portem. Dlatego nic ważnego nie może zależeć od pamięci przeglądarki — trwałe ustawienia idą na serwer.

19. Panel deweloperski

Przycisk „Logi” (`#btn-logs`) pokazuje się tylko, gdy `GET /api/config` zwraca `dev=true`. Panel odpytuje `/api/logs` co 1,5 s, ma auto-przewijanie, kopiowanie i czyszczenie. Cały kod jest między znacznikami `DEV:START` / `DEV:END` i znika z wydania publicznego.

20. Jak rozszerzać interfejs

1. Nowy element DOM dodaj w `index.html` z atrybutem `data-i18n` i domyślnym polskim tekstem; klucz w czterech językach w `i18n.js`.
2. Uchwyt do elementu dopisz do obiektu `els` na początku `app.js`; nasłuchuj rejestruj raz, na końcu odpowiedniej sekcji.
3. Wywołania serwera przez `fetch`; błędy pokazuj przez `toast(backendErrorMessage(dane, r.statusText), "error")`.
4. Jeśli element ma się znaleźć w samouczku, dodaj selektor do `TARGETS` i krok do `STEPS` oraz teksty `onboarding.steps.<id>`; nową funkcję dla istniejących użytkowników ogłoś przez `registerFeatureHint`.
5. Kolory tylko przez zmienne CSS. Sprawdź oba motywy.
6. Sprawdź stronę w trybie serwera (`KEEPCLIP_SERVER_ONLY=1`, port 8770) w zwykłej przeglądarce — most okna wtedy nie istnieje, a kod musi to znosić (`callWindowApi`).
7. Uruchom `node tests/i18n-check.js` i testy backendu, jeśli zmieniło się API.